
PHASE-BY-PHASE IMPLEMENTATION GUIDE

FreightQuote AI

What Each Phase Builds — Models, Frontend Wiring, and API Design

DOCUMENT TYPE	Implementation guide — per phase: collections, form-to-API connection, endpoint specs
PROJECT CODE	FQ-AMB-001 / PHASES
COVERS	All 4 milestones, 8 weeks
DATABASE	MongoDB 7.x
READ FIRST	Sections 2 and 3 explain the connection pattern and API shape. Every phase then applies the same pattern.

Table of Contents

1. The Four Phases in Plain English

What each one delivers and why in that order

2. How the Frontend Connects to the Backend

The one pattern used everywhere

3. How to Write an API Endpoint

Layering, validation, envelopes, errors

4. Phase 1 — Route Intelligence

Collections, pages, endpoints

5. Phase 2 — Pricing & Margin

Collections, pages, endpoints

6. Phase 3 — Weather & Customs

Collections, pages, endpoints

7. Phase 4 — Intelligence Engine & Deployment

Collections, pages, endpoints

8. Quick Reference

All collections and endpoints by phase

1. The Four Phases in Plain English

Phase	Name	What it actually means
1 Wk 1-2	Route Intelligence & Quotation Foundation	The system learns to answer: " <i>How does cargo get from A to B, and how long does it take?</i> " A customer enters an enquiry, the system finds which shipping services connect those two ports, calculates the distance and the transit time, ranks the options, and shows a rough price. Nothing here is a real price yet — it is a placeholder so the screen is not empty.
2 Wk 3-4	Pricing Intelligence & Margin Optimisation	The system learns to answer: " <i>What does this actually cost us, and what should we charge?</i> " Real rate cards replace the placeholder. Every charge becomes a separate line: base freight, fuel, terminal handling, documentation. Then a margin is added, checked against a floor, and routed for approval if it breaches.
3 Wk 5-6	Weather Risk & Customs Intelligence	The system learns to answer: " <i>What could go wrong?</i> " Weather along the route, storm risk, delay probability. Customs rules for the origin and destination country, the documents needed, and whether the commodity is restricted. Each shipment gets a risk score with a visible breakdown.
4 Wk 7-8	Freight Intelligence Engine & Deployment	The system learns to answer: " <i>Putting all of that together, what do you recommend and why?</i> " All five agents' findings are fused into one recommendation with a written explanation. Executive dashboards, full testing, production deployment.

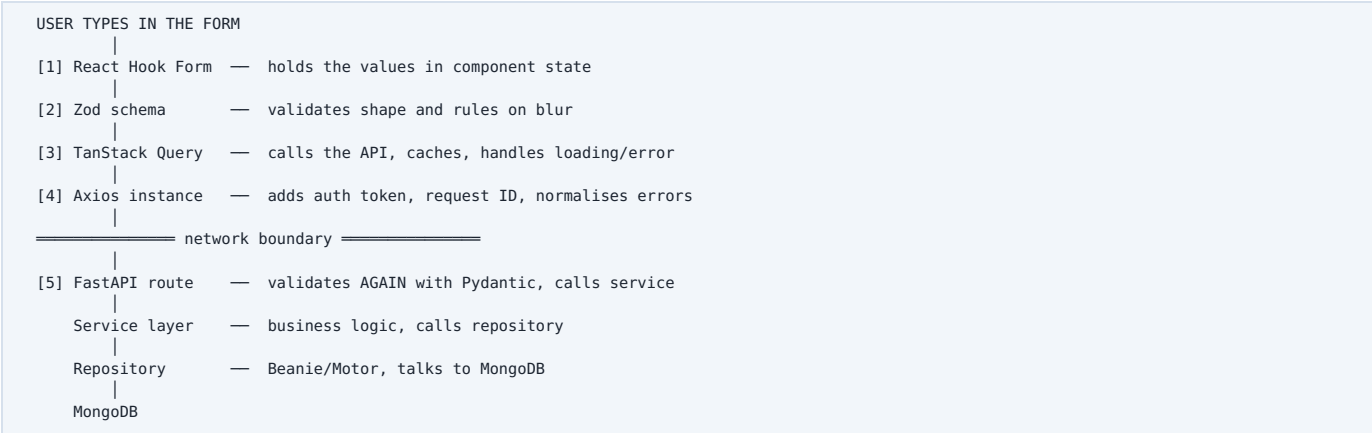
Why this order and not another

Every phase depends on the one before it. You cannot price a route you have not found. You cannot assess weather on a route that does not exist. You cannot fuse findings that have not been produced. Phase 1 being "just routes" feels slow, but it is the foundation the other three stand on — and if the port and distance data is wrong in Phase 1, every number in Phases 2, 3 and 4 is wrong too.

2. How the Frontend Connects to the Backend

This is one pattern, used identically on every screen in every phase. Learn it once.

2.1 The five layers

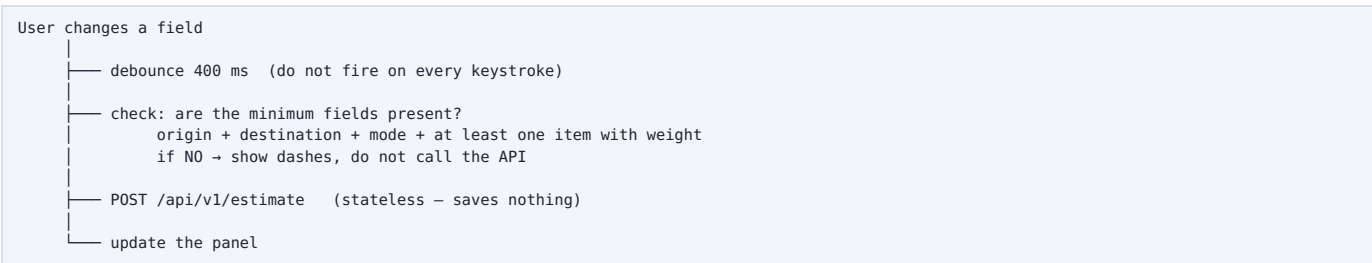


2.2 Two things people get wrong here

Mistake	Why it breaks
Trusting frontend validation	Zod runs in the browser. Anyone can open DevTools and post whatever they want to your API. The backend must validate independently with Pydantic. Frontend validation is for user experience, never for security.
Putting API calls inside page components	The same call gets copy-pasted into three pages, then the endpoint changes and two of them break. All API functions live in features/<feature>/api/, wrapped in a hook. Pages only call hooks.

2.3 The Live Estimate panel — a special case

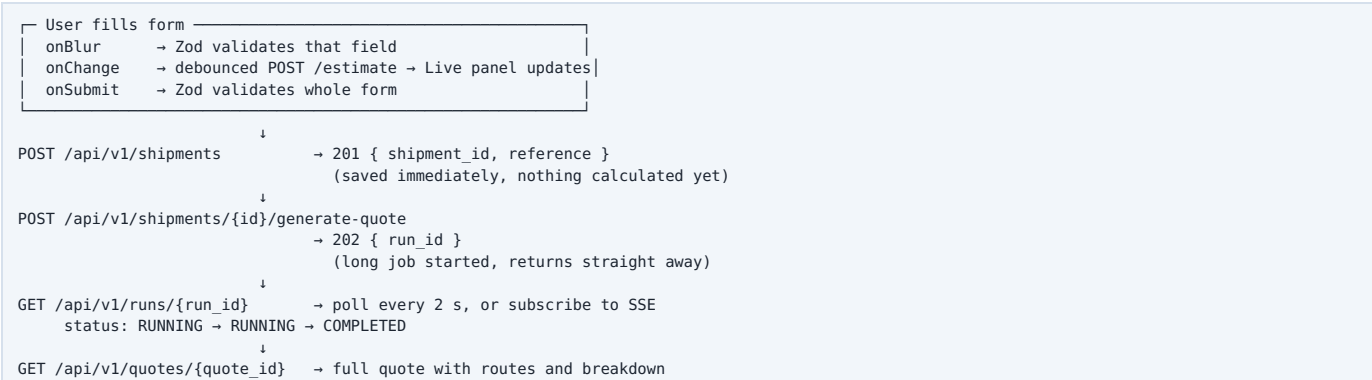
Every other screen fetches data and shows it. The Live Estimate is different: it must update as the user types, before anything is saved.



Two design points that matter:

- **Debounce, do not throttle.** Fire once after the user stops typing for 400 ms. Throttling fires during typing and produces flickering half-computed estimates.
- **Cancel the previous request.** If a new estimate fires while an old one is in flight, abort the old one. Otherwise a slow earlier response can land after a fast later one and overwrite the panel with stale numbers. TanStack Query handles this if you key the query on the form values.

2.4 The full journey for one enquiry



↓
Navigate to `/app/quotes/{id}`

Never make the user wait on a long request

Quote generation calls five agents and several external APIs. It takes 20–60 seconds. If you make that a synchronous POST, the browser times out, proxies drop the connection, and the user sees nothing. Return `202 Accepted` with a `run_id` immediately, then poll or stream. This is why `agent_runs` exists as a collection.

3. How to Write an API Endpoint

3.1 Three layers, always

Layer	File	Responsibility — and what it must NOT do
Route	api/v1/routes/shipments.py	Parse and validate the request, check permissions, call one service function, shape the response. No business logic. No database calls.
Service	services/shipment_service.py	All business rules: validation beyond shape, calculations, orchestration, transactions. No HTTP concepts — no request, response, or status codes here.
Repository	repositories/shipment_repo.py	Database queries only. No business rules. Returns documents or None.

The test: you should be able to call a service function from a script, a test, or a background worker without any HTTP involved. If it needs a request object, the layering is wrong.

3.2 The standard endpoint shape

```
@router.post("/shipments", status_code=201, response_model=ApiResponse[ShipmentOut])
async def create_shipment(
    payload: ShipmentCreateIn,          # Pydantic validates the body
    user: User = Depends(get_current_user), # auth
    idem_key: str = Header(..., alias="Idempotency-Key"),
):
    shipment = await shipment_service.create(payload, user, idem_key)
    return ApiResponse(success=True, data=ShipmentOut.from_doc(shipment))
```

Four lines. Everything hard happens inside `shipment_service.create()`. That is the target for every endpoint you write.

3.3 Response envelopes — agree these on day one

```
// SUCCESS
{
  "success": true,
  "data": { ... },
  "meta": { "request_id": "req_8f3a", "next_cursor": "eyJ..." }
}

// ERROR
{
  "success": false,
  "error": {
    "code": "PORT_PAIR_UNSERVICED",
    "message": "No carrier service connects INNSA to PECLL for OCEAN mode.",
    "details": [ { "field": "destination_gateway", "issue": "no_service" } ],
    "request_id": "req_8f3a"
  }
}
```

The frontend switches on `error.code`, never on `error.message`. Messages get reworted; codes are a contract.

3.4 Error codes to define now

Code	HTTP	Meaning
VALIDATION_FAILED	400	Body failed schema validation
UNAUTHENTICATED	401	Missing or expired token
FORBIDDEN	403	Role lacks permission
NOT_FOUND	404	Resource does not exist or is out of scope
PORT_PAIR_UNSERVICED	422	No carrier service links those gateways
LANE_DISTANCE_MISSING	422	Port distance not seeded for that lane
INVALID_TRANSITION	409	Illegal state machine move
QUOTE_BELOW_MARGIN_FLOOR	409	Phase 2 — margin under policy
RATE_CARD_EXPIRED	422	Phase 2 — no valid rate
COMPLIANCE_SIGNOFF_REQUIRED	409	Phase 3 — customs blocks issuance
RATE_LIMITED	429	Too many requests

3.5 Rules that apply to every endpoint

Rule	Detail
Money as strings	{ "amount": "384500.0000", "currency": "INR" }. Never a JSON number — JavaScript parses it as a float and destroys the precision you protected in MongoDB.
Cursor pagination	?limit=25&cursor=eyJ... Never skip() — it degrades linearly as the collection grows.
Idempotency	Idempotency-Key header required on all creates. Store keys in Redis 24 h; a repeat returns the original response instead of creating a duplicate.
Request ID	Generated at the edge, echoed in every response and log line, passed to the agent service.
Scoping	Customer-role queries filter on customer_id from the token , never from a query parameter.
Data freshness	Any response built on external data carries data_as_of timestamps so the UI can show how old it is.

4. Phase 1 — Route Intelligence

PHASE 1 · WEEKS 1-2

Goal: an enquiry produces ranked route options with transit times and an indicative total.

4.1 Collections created in this phase

Collection	Type	Purpose
users	Master	Login, roles, customer scoping
customers	Master	Company records with tier and contacts embedded
ports	Master	Gateways with GeoJSON location and a 2dsphere index
port_distances	Master	Seeded sea distances — top 50 lanes
carriers	Master	Shipping lines with reliability scores
carrier_services	Master	Port rotations — the graph routes are found in
container_types	Master	20GP, 40HC, payload limits
business_calendars	Master	Working days and holidays per country
lane_rates	Stub	Flat regional rates — replaced entirely in Phase 2
shipments	Transactional	The enquiry, with items embedded
routes	Transactional	Generated options with legs, transit breakdown, scores, geometry
freight_quotes	Transactional	Quote identity and current version pointer
quote_versions	Transactional	Immutable priced document with the route snapshotted
counters	System	Atomic reference number generation
activity_logs	System	Append-only audit trail

4.2 Frontend in this phase

Page	Talks to	Connection notes
/login	POST /auth/login	Store access token in memory (Zustand), refresh token in httpOnly cookie. Axios interceptor auto-refreshes on 401 and retries once.
/ship	GET /gateways/search POST /estimate POST /shipments	Gateway dropdown is async — debounce 300 ms, minimum 2 characters. Live Estimate debounces 400 ms and aborts in-flight requests. Submit creates then triggers generation.
/quotes	GET /quotes	Filters go in the URL search params, not component state, so the view is shareable and back-button safe. Query key includes the filters so TanStack caches per filter set.
/quotes/{id}	GET /quotes/{id} GET /shipments/{id}/routes	Two parallel queries. Map renders routes[].geometry directly — it is already GeoJSON, so MapLibre consumes it without conversion.
/routes	GET /routes/performance GET /ports/congestion	Read-only dashboard. Cache for 5 minutes — this data does not change by the second.

4.3 APIs in this phase

Method	Endpoint	Request → Response
POST	/api/v1/auth/login	{email, password} → {access_token, user} + refresh cookie
POST	/api/v1/auth/refresh	refresh cookie → new {access_token}
GET	/api/v1/auth/me	→ {id, name, role, customer_id, permissions[]}
GET	/api/v1/gateways/search	?q=mumbai&mode=OCEAN → max 10 {code, name, city, country, type}
GET	/api/v1/masterdata/container-types	→ list for the dropdown
POST	/api/v1/estimate	Partial form state → {weights, transit_range, indicative_total, route_count}. Stateless, saves nothing.
POST	/api/v1/shipments	Full form → 201 {id, reference, status}
GET	/api/v1/shipments	Filters + cursor → paginated list
GET	/api/v1/shipments/{id}	→ shipment with items and linked quotes

POST	/api/v1/shipments/{id}/generate-quote	→ 202 {run_id}
GET	/api/v1/runs/{run_id}	→ {status, progress, quote_id?} — poll target
GET	/api/v1/shipments/{id}/routes	→ ranked routes with legs, transit, scores, geometry
GET	/api/v1/quotes	Filters + cursor → paginated list
GET	/api/v1/quotes/{id}	→ quote + current version (customer projection if CUSTOMER role)
GET	/api/v1/routes/performance	?lane= → on-time percentage by lane and carrier
GET	/api/v1/ports/congestion	→ waiting hours per port

Worked example — the estimate endpoint

```
POST /api/v1/estimate
{
  "origin_gateway": "INNSA",
  "destination_gateway": "AEJEA",
  "mode": "OCEAN",
  "load_type": "FCL",
  "incoterm": "FOB",
  "ready_date": "2026-08-12",
  "items": [
    { "package_type": "CONTAINER", "container_type": "40HC",
      "container_count": 2, "gross_weight_kg": "18400.000" }
  ]
}

→ 200
{
  "success": true,
  "data": {
    "charge_basis": { "type": "PER_CONTAINER", "units": "2",
      "label": "2 x 40HC" },
    "distance": { "value": 1205, "unit": "nm" },
    "transit": { "min_days": 6, "max_days": 10,
      "estimated_arrival": "2026-08-22" },
    "indicative_total": { "amount": "384500.0000", "currency": "INR" },
    "is_indicative": true,
    "route_count": 3
  },
  "meta": { "request_id": "req_8f3a" }
}
```

5. Phase 2 — Pricing & Margin

PHASE 2 · WEEKS 3-4

Goal: replace the placeholder price with a real, itemised cost build-up plus margin and approval.

5.1 Collections added

Collection	Purpose
rate_cards	Contract and spot rate cards with validity windows
rate_card_lines	Per lane and container type: base rate, minimum charge
surcharges	BAF, CAF, THC, ISPS, PSS — with calculation type and applicability rules
margin_policies	Floor, target and stretch margin by lane, customer tier and cargo type
approval_rules	Ordered rules mapping a breach condition to an approver role
quote_approvals	Approval requests and decisions with reasons
fx_rates	Daily rates, frozen onto each quote version

5.2 Changes to existing collections

Collection	Change
quote_versions	pricing.cost_components[] is populated. total_cost, margin_amount, margin_pct filled. is_indicative becomes false. rate_card_id frozen. Because you designed pricing as a nested object in Phase 1, no migration is needed — the new fields simply appear on new documents.
lane_rates	Deprecated. Keep it read-only for a release so old quotes remain explainable, then archive.

5.3 Frontend in this phase

Page	Connection notes
/app/quotes/builder/:id	The core screen. Margin slider calls PATCH /quotes/{id}/margin on release, not on drag. On a 409 QUOTE_BELOW_MARGIN_FLOOR, show the floor warning and reveal the "Submit for approval" button.
/app/quotes/:id	Adds the cost breakdown table and version timeline. Each version is a separate fetch — they are immutable, so cache them indefinitely.
/app/quotes/approvals	Approver queue. Approve and reject both require a reason on reject.
/app/pricing/rate-cards	Import flow is two steps: upload returns a validation report without committing; a second call commits. Never commit a rate card on upload — one bad file corrupts every quote until someone notices.
/portal/quotes/:id	Customer view. Uses the customer projection — cost, margin and buy-rate keys are stripped in the backend.

5.4 APIs added

Method	Endpoint	Purpose
GET	/api/v1/pricing/rate-cards	List with validity windows
POST	/api/v1/pricing/rate-cards/validate	Upload CSV/XLSX → validation report, no commit
POST	/api/v1/pricing/rate-cards/commit	Commit a previously validated upload
GET	/api/v1/pricing/cost-breakdown	?shipment_id=&route_id= → itemised components
POST	/api/v1/quotes	Create a quote from a shipment and selected route
POST	/api/v1/quotes/{id}/versions	New immutable version; previous becomes SUPERSEDED
PATCH	/api/v1/quotes/{id}/margin	Adjust margin. 409 if below floor without authority.
POST	/api/v1/quotes/{id}/submit-approval	Route to the approver resolved by the rule set
POST	/api/v1/quotes/{id}/approve	Approve with optional comment
POST	/api/v1/quotes/{id}/reject	Reject with mandatory reason
POST	/api/v1/quotes/{id}/issue	Generate PDF, email customer, start validity timer
GET	/api/v1/quotes/approvals/queue	Pending approvals with breach reasons
GET	/api/v1/admin/margin-policies	Read and update floors and targets

The margin floor is enforced in the backend, never in the slider

The React slider showing a red warning is user experience. The `PATCH /margin` endpoint returning 409 is the control. If the check lives only in the frontend, anyone can post a margin below floor with curl and it will be accepted.

6. Phase 3 — Weather & Customs

PHASE 3 · WEEKS 5-6

Goal: every quote carries a weather assessment, a customs checklist, and a scored risk breakdown.

6.1 Collections added

Collection	Purpose
weather_risks	Per-route assessment: condition, risk score, delay probability, storm alerts
shipment_risk_scores	Composite score with each factor's contribution stored separately
alerts	Risk alerts with severity, acknowledge and resolve workflow
customs_requirements	Per-shipment requirement list with compliance status
document_requirements	Checklist items, provided or missing
compliance_checks	Readiness score, risk flags, officer sign-off
regulations	Regulation documents with source URL and verification date
regulation_chunks	Chunked text with embedding vectors — Atlas Vector Search index goes here
hs_codes	Tariff classifications and restrictions
port_congestion_snapshots	Time-series collection, monthly partitions
data_freshness	Per-provider last-success timestamps — drives staleness warnings

6.2 The vector search index

```
// Atlas Vector Search index on regulation_chunks
{
  "fields": [
    { "type": "vector", "path": "embedding",
      "numDimensions": 1024, "similarity": "cosine" },
    { "type": "filter", "path": "country" },
    { "type": "filter", "path": "is_current" }
  ]
}

// Query
db.regulation_chunks.aggregate([
  { $vectorSearch: {
    index: "regulation_vector_index",
    path: "embedding",
    queryVector: [...],
    numCandidates: 100,
    limit: 5,
    filter: { country: "AE", is_current: true }
  }},
  { $project: { content: 1, regulation_id: 1,
    score: { $meta: "vectorSearchScore" } }}
])
```

This is where the MongoDB choice pays off — no separate vector database to run, back up or keep in sync.

6.3 Frontend in this phase

Page	Connection notes
/app/risk	Risk dashboard. Every score shows its factor contributions — a number without a breakdown is not usable by a broker.
/app/risk/weather	Weather overlay on the route map. Show data_as_of prominently; stale weather is worse than none.
/app/customs/check/:id	Checklist with each requirement linked to its regulation citation. Sign-off button is only enabled for COMPLIANCE_OFFICER.
/app/customs/regulations	Corpus browser with semantic search and freshness indicators per document.

6.4 APIs added

Method	Endpoint	Purpose
GET	/api/v1/risk/shipments/{id}	Composite score with itemised factors
GET	/api/v1/risk/weather	?route_id= → assessment and delay probability

GET	/api/v1/risk/alerts	Alert inbox
POST	/api/v1/risk/alerts/{id}/acknowledge	Acknowledge an alert
POST	/api/v1/customs/check	Run compliance validation for a shipment
GET	/api/v1/customs/requirements	?shipment_id= → checklist with citations
POST	/api/v1/customs/signoff/{check_id}	Officer sign-off. COMPLIANCE_OFFICER only.
POST	/api/v1/customs/regulations/search	Semantic search over the corpus
POST	/api/v1/customs/regulations/ingest	Ingest documents, chunk, embed
GET	/api/v1/integrations/health	Per-provider status and data freshness

The customs endpoint must be able to say "I don't know"

If the regulation corpus has no coverage for a country or commodity, POST /customs/check returns readiness: INSUFFICIENT_COVERAGE with an empty requirement list — not a guessed checklist. A confident wrong customs answer costs a customer a penalty and costs you the account.

7. Phase 4 — Intelligence Engine & Deployment

PHASE 4 · WEEKS 7-8

Goal: fuse all agent findings into one explained recommendation; ship to production.

7.1 Collections added

Collection	Purpose
agent_runs	One document per quote generation: status, duration, tokens, cost
agent_steps	Per-agent input, output, data sources, model version, latency
daily_quote_rollups	Pre-aggregated daily metrics by lane, carrier, tier
lane_performance_rollups	Volume, transit, on-time, margin, win rate per lane

Dashboards read **only** from rollups, never aggregating raw quote_versions live. A nightly job builds them. This is the difference between a dashboard that loads in 200 ms and one that times out at 12 months of data.

7.2 Frontend in this phase

Page	Connection notes
/app/executive	Reads rollups. Cache 15 minutes — executives do not need second-level freshness.
/app/agents/runs/:id	Full trace: every agent's inputs, outputs and timings. This is the "why did it recommend that?" screen and it is what earns broker trust.
/app/admin/*	Margin policy, approval rules, agent settings, integrations health, audit log.

7.3 APIs added

Method	Endpoint	Purpose
GET	/api/v1/agents/runs	Run list with status and duration
GET	/api/v1/agents/runs/{id}	Full step-by-step trace
GET	/api/v1/agents/runs/{id}/stream	SSE progress during generation
POST	/api/v1/agents/runs/{id}/retry	Resume from last checkpoint
GET	/api/v1/analytics/kpi	Headline tiles from rollups
GET	/api/v1/analytics/lanes	Lane performance
GET	/api/v1/analytics/forecast	Pipeline-weighted revenue forecast
POST	/api/v1/analytics/reports/export	Queue CSV/PDF export job
GET	/api/v1/audit/logs	Filterable append-only log

8. Quick Reference

8.1 Collections by phase

Phase	New	Collections
1	15	users, customers, ports, port_distances, carriers, carrier_services, container_types, business_calendars, lane_rates, shipments, routes, freight_quotes, quote_versions, counters, activity_logs
2	7	rate_cards, rate_card_lines, surcharges, margin_policies, approval_rules, quote_approvals, fx_rates
3	11	weather_risks, shipment_risk_scores, alerts, customs_requirements, document_requirements, compliance_checks, regulations, regulation_chunks, hs_codes, port_congestion_snapshots, data_freshness
4	4	agent_runs, agent_steps, daily_quote_rollups, lane_performance_rollups

8.2 Endpoint count by phase

Phase	New endpoints	Focus
1	16	Auth, master data, enquiry, routes, indicative quote
2	13	Rate cards, cost build-up, margin, approval, issuance
3	9	Weather, risk, customs, regulation search
4	9	Agent traces, analytics, audit

8.3 The five things to get right in Phase 1 so nothing breaks later

#	Decision
1	Every monetary field is Decimal128 in Mongo and a string over the API. Retrofitting this means a data migration and a frontend rewrite.
2	quote_versions.pricing is a nested object, not flat fields. Phase 2 adds keys inside it with zero migration.
3	Quote versions are insert-only, and they snapshot the route rather than referencing it.
4	Long jobs return 202 + run_id. Phases 3 and 4 make runs much longer; the pattern must already exist.
5	Customer scoping comes from the token, and the customer projection lives in the backend with a test asserting no cost keys leak.